

Métrique de similarité perceptuelle-physique pour la synthèse audio

Restitution finale

Léon Albert - Thomas Perrier

Table des matières

1	Introduction	3
2	Comparaison des mesures de distances	4
2.1	Méthodologie	4
2.2	Espace des paramètres	4
2.3	Fonction de perception	4
2.4	Algorithme de recherche des plus proches voisin	4
2.5	Méthode Bruteforce	4
2.6	Méthode PNP	4
2.6.1	Métrique Riemannienne	4
2.6.2	Résolution algorithmique	5
2.7	Résultats	5
2.7.1	Notations	5
2.7.2	Choix de K	5
2.7.3	Comparaison des méthodes	6
2.7.4	Exemples audio	6
3	Réalisation d'un graphe des plus proches voisins	7
3.1	Motivation	7
3.2	Méthode des hubs	7
3.2.1	Distance PNP avec hubs	7
3.2.2	Choix des hubs	8
3.3	Création du graphe	10
3.4	Caractéristiques du graphe	10
3.5	Clustering	11
4	Applications	13
4.1	Chemin entre deux sons de référence	13
4.2	Marche aléatoire	13
5	Conclusion sur les résultats finaux	15

1 Introduction

L’objectif de ce projet de concevoir un algorithme permettant, à partir des paramètres physiques d’un synthétiseur de sons de percussion, de retrouver de nouveaux paramètres qui mènent à un son proche du son de référence selon la perception humaine.

Ce travail s’aligne dans le prolongement de l’étude de Han Han, Vincent Lostalen et Mathieu Lagrange [1] qui visait à réaliser le parcours inverse du synthétiseur : en partant d’un son retrouver des paramètres physique qui permettent de le reproduire. Une solution basée sur un réseau de neurones ne peut produire qu’une unique proposition de paramètres pour un son donné, ce qui va motiver l’objectif de ce projet pour palier à cette limitation.

Nous nous appuyons sur le même synthétiseur à modèle physique de son de percussion (g) utilisé par ces derniers [2]. Le choix de la fonction de perception (Φ) est initialement laissé ouvert.

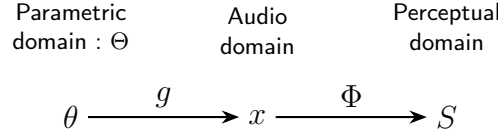


FIGURE 1 – Architecture du projet

Afin de trouver des jeux de paramètres proches nous avons défini plusieurs mesures de distance basées sur les différents domaines :

- **P-Loss** : la méthode très naïve qui consiste à effectuer une distance euclidienne sur les paramètres pris en entrée du modèle $\theta \in \Theta$.
- **Bruteforce** : la méthode naïve qui consiste à effectuer une distance euclidienne sur la sortie du modèle S , cette méthode calculant la distance exacte dans le domaine perceptuel servira de vérité terrain pour les comparaisons.
- **PNP** : la méthode basé sur la forme quadratique $\langle \tilde{\theta} - \theta \mid M(\theta) \mid \tilde{\theta} - \theta \rangle$ avec M la métrique Riemannienne associée à $(\Phi \circ g)$ tel qu’utilisé dans les travaux précédents [3].

Dans un premier temps l’objectif du projet est donc pour chaque méthodes de calcul de distance d’obtenir pour des paramètres θ les K plus proches voisins, afin de pouvoir comparer les différentes méthodes selon des critères de précision et de vitesse de calcul, nous espérons ainsi pouvoir valider l’utilisation de la mesure PNP pour la deuxième partie du projet.

Après cette première phase, l’objectif est donc d’utiliser la mesure PNP pour constituer le graphe complet des K plus proches voisins, à partir duquel de nombreuse applications pourrons émerger : transformation continue du son, regroupement par similarités sonores, etc...

2 Comparaison des mesures de distances

2.1 Méthodologie

Le problème se décompose en plusieurs étapes :

- Réaliser une discrétisation de l'espace des paramètres, permettant de délimiter l'espace de recherche.
- Choix d'un point de référence θ (par exemple le premier dans le dataset)
- Calculer les K plus proches voisins de θ pour chaque mesures de distance
- Comparer les voisinages obtenus par chaque mesures ainsi que le temps de calcul

2.2 Espace des paramètres

Dans la phase de cadrage du projet nous avons défini l'espace des paramètres sur lequel la recherche est effectuée. Cet espace comporte 5 dimensions correspondantes à 5 paramètres physiques tels que la taille, la rigidité de la peau, la tension... Nous avons choisi les bornes définies par les travaux nous précédant puis nous définissons une subdivision de 10 pour chaque paramètres, ce qui correspond à une subdivision classique de synthétiseur. Cela nous mène donc à un ensemble de 10^5 possibilités de jeux de paramètres ce qui explique la difficulté computationnelle du projet.

2.3 Fonction de perception

Plusieurs fonctions de perception sont détaillées dans la littérature, l'unique critère initial pour notre projet étant la différentiabilité, nous avons sélectionné le Joint time-frequency scattering (JTFS) [4], [5]

2.4 Algorithme de recherche des plus proches voisin

L'algorithme de recherche des voisins est un algorithme classique de K-nn : pour un point de référence on calcule sa distance aux autres points du jeu de données, on ordonne ensuite ces points dans l'ordre croissant des distances pour conserver ses K plus proches voisins.

2.5 Méthode Bruteforce

Pour la méthode de bruteforce nous calculons les distances euclidiennes dans l'espace de perception, il est donc nécessaire de calculer l'ensemble des $S(\theta)$.

Cela est réalisé en amont, en particulier à cause de la taille des résultats (50go pour une sous-division des paramètres de 10), mais pour la comparaison de la vitesse de calcul des différentes méthodes nous devons évidemment en prendre compte.

2.6 Méthode PNP

2.6.1 Métrique Riemannienne

Pour définir la mesure PNP nous partons de la distance euclidienne dans l'espace de perception :

$$d_{\Phi}(\theta, \tilde{\theta}) = \|\Phi \circ g(\tilde{\theta}) - \Phi \circ g(\theta)\|^2 = \langle \Phi \circ g(\tilde{\theta}) - \Phi \circ g(\theta) | \Phi \circ g(\tilde{\theta}) - \Phi \circ g(\theta) \rangle \quad (1)$$

Avec un développement de Taylor :

$$\Phi \circ g(\tilde{\theta}) \approx \Phi \circ g(\theta) + \nabla_{\Phi \circ g}(\theta) \cdot (\tilde{\theta} - \theta) \quad (2)$$

Et donc en combinant 1 et 2 on obtient :

$$d_{\Phi}(\theta, \tilde{\theta}) \approx \langle \nabla_{\Phi \circ g}(\theta) \cdot (\tilde{\theta} - \theta) | \nabla_{\Phi \circ g}(\theta) \cdot (\tilde{\theta} - \theta) \rangle = (\tilde{\theta} - \theta)^T \nabla_{\Phi \circ g}(\theta)^T \nabla_{\Phi \circ g}(\theta) (\tilde{\theta} - \theta) \quad (3)$$

Et on définit ainsi $M(\theta) = \nabla_{\Phi \circ g}(\theta)^T \nabla_{\Phi \circ g}(\theta)$ tel que :

$$d_{\Phi}(\theta, \tilde{\theta}) \approx (\tilde{\theta} - \theta)^T M(\theta) (\tilde{\theta} - \theta) \quad (4)$$

2.6.2 Résolution algorithmique

Afin de pouvoir utiliser cette mesure pour calculer les distances à θ nous avons besoin de $M(\theta)$. Les fonctions Φ et g sont bien trop complexes pour obtenir une expression littérale, nous nous sommes donc tournés vers les fonctions d'autograd de PyTorch.

En vu des dimensions des différents espaces (4 pour θ puis de l'ordre de 10^4 pour x et enfin de l'ordre de 10^5 pour S) la stratégie de calcul à choisir est le mode forward, pour cela nous avons testé deux fonctions d'autograd présentes dans PyTorch, le mode forward étant beaucoup moins utilisé dans le machine learning une seule des deux est suffisamment complète pour fonctionner dans notre cas : `torch.func.jacfwd()`

2.7 Résultats

2.7.1 Notations

Pour évaluer et comparer les performances d'exactitude des différentes méthode nous introduisons quelques notations :

- $\mathcal{V}_{P-loss,K}^{(i)}$: L'ensemble des K -plus proches voisins du point $\theta^{(i)}$ obtenus avec la méthode P-Loss. Aussi appelé voisinage du point $\theta^{(i)}$ pour K voisins selon la méthode P-Loss.
- $v_{P-loss,l}^{(i)}$: Le l -ième voisin du point $\theta^{(i)}$ obtenu avec la méthode P-Loss.
- On définit de même $\mathcal{V}_{PNP,K}^{(i)}, \mathcal{V}_{Brute force,K}^{(i)}$ et $v_{PNP,l}^{(i)}, v_{Brute force,l}^{(i)}$

2.7.2 Choix de K

Nous avons tout d'abord pensé à utiliser la métrique IoU (*Intersection Over Union*). Cette métrique est définie comme le rapport entre le cardinal de l'intersection du voisinage d'un point de référence donné par la méthode à évaluer et la méthode de référence (Brute force), et le cardinal de l'union de ces deux ensembles (Equation 5 et figure 2).

$$IoU_{Methode,K}^{(i)} = \frac{|\mathcal{V}_{Methode,K}^{(i)} \cap \mathcal{V}_{Brute force,K}^{(i)}|}{|\mathcal{V}_{Methode,K}^{(i)} \cup \mathcal{V}_{Brute force,K}^{(i)}|} \quad (5)$$

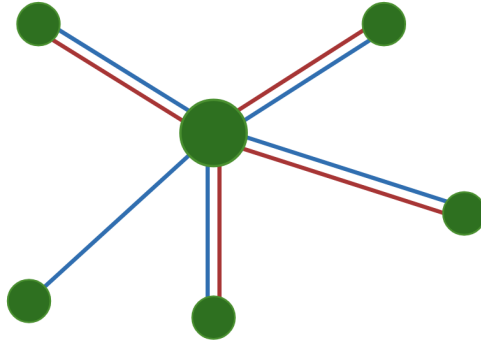


FIGURE 2 – Exemple de voisinages (rouge et bleu) pour illustrer la métrique IoU, dans cet exemple l'union des voisinages possède 5 points et l'intersection 4. On obtient donc $\mathbf{IoU} = \frac{4}{5}$.

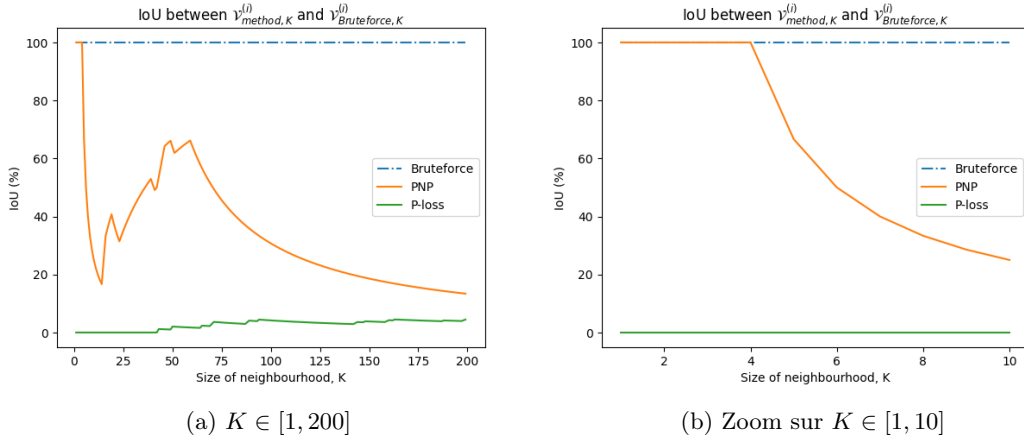


FIGURE 3 – Critère de l’IoU en fonction de la taille K du voisinage : pour $K < 5$ le voisinage de la méthode PNP est exactement celui de référence, pour $40 < K < 60$ nous avons un compromis entre la taille du voisinage et la précision de celui ci.

2.7.3 Comparaison des méthodes

Une fois le choix de K fixé entre 30 et 70 (par exemple 50) et de la sous-division de l’espace des paramètres, on peut comparer les différentes méthodes vis-à-vis de deux critères : $IoU_{Methode, K}$ (%) et le temps de calcul d’un voisinage noté $T_{Methode, K}$ (s) (Figure 4).

On différencie la méthode Bruteforce avec et sans le pré-calcul des $S(\theta)$, car dépendamment des applications Celui-ci pourrait ou non être imposé à l’utilisateur. Et dans ce cas l’impacte sur le temps de calcul est important.

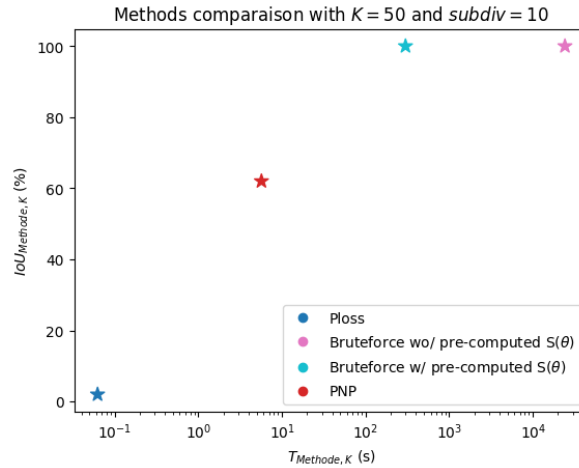


FIGURE 4 – Comparaison des méthodes pour le calcul du voisinage d’un point selon le temps de calcul et le critère de l’IoU : la méthode PNP se place en compromis entre les deux autres méthodes avec un temps de calcul à une échelle utilisateur (quelques dizaines de secondes) bien inférieur à la méthode Bruteforce et une précision largement supérieure à la méthode P-loss (environ 60%)

2.7.4 Exemples audio

Des exemples audio peuvent être écoutés sur cette page : leon-albert.github.io/perceptual-kNN/

3 Réalisation d'un graphe des plus proches voisins

3.1 Motivation

Après avoir calculé les distances entre tous les points du jeu de données et ainsi retrouvé les plus proches voisins de chaque point, au sens de la perception humaine, il nous semble intéressant de réaliser un graphe à partir de ces informations. Ce graphe nous permettrait par exemple de passer d'un son à un autre de manière itérative par un parcours de graphe. On réalise ainsi une séquence audio partant d'un son particulier vers un autre tout en conservant une continuité entre les sons joués. Il serait aussi possible à partir d'un son de référence, de donner à l'utilisateur du synthétiseur des sons proches à l'aide d'un parcours de graphe différent.

3.2 Méthode des hubs

3.2.1 Distance PNP avec hubs

Pour construire le graphe avec la distance PNP nous avons besoin de calculer les voisins de chaque points et donc l'ensemble des $M(\theta)$ ce qui reviendrait plus ou moins à un temps de calcul équivalent à calculer l'ensemble des $S(\theta)$. Pour palier à ce problème nous choisissons d'effectuer une approximation supplémentaire :

On défini à partir de (4) la fonction suivante pour tout $\theta_1, \theta_2, \theta_3 \in \Theta$:

$$d_{PNP}(\theta_1, \theta_2, \theta_3) = (\theta_3 - \theta_1)^T M(\theta_2)(\theta_3 - \theta_1) \quad (6)$$

Et on fait l'hypothèse que pour tout $\theta, \tilde{\theta} \in \Theta$ assez proche alors :

$$M(\theta) \approx M(\tilde{\theta}) \quad (7)$$

Nous introduisons ensuite plusieurs notations :

- $\mathcal{H} \subset \Theta$ avec $h \in \mathcal{H}$ les hubs
- $\mathcal{A} : \Theta \rightarrow \mathcal{H}$ l'allocation des hubs tel que $\forall \theta \in \Theta, \mathcal{A}(\theta) = \arg \min_{h \in \mathcal{H}} d_{PNP}(\theta, h, h)$
- $\mathcal{R} : \mathcal{H} \rightarrow \Theta$ la région des hubs tel que $\forall h \in \mathcal{H}, \mathcal{R}(h) = \{\theta \in \Theta \text{ tel que } \mathcal{A}(\theta) = h\}$

De cette manière avec un $\mathcal{H}$ suffisamment bien choisi et les $\mathcal{A}$ et $\mathcal{R}$ associés on peut déduire de (4) et (7) que pour tout $h \in \mathcal{H}$ et $\theta, \tilde{\theta} \in \mathcal{R}(h)$:

$$d_{\Phi}(\theta, \tilde{\theta}) \approx d_{PNP}(\theta, h, \tilde{\theta}) = (\tilde{\theta} - \theta)^T M(h)(\tilde{\theta} - \theta) \quad (8)$$

Et nous choisissons pour tout $h, \tilde{h} \in \mathcal{H}$ et $\theta \in \mathcal{R}(h), \tilde{\theta} \in \mathcal{R}(\tilde{h})$:

$$d_{\Phi}(\theta, \tilde{\theta}) := \frac{1}{2}(d_{PNP}(\theta, h, \tilde{\theta}) + d_{PNP}(\theta, \tilde{h}, \tilde{\theta})) \quad (9)$$

Donc nous pouvons finalement définir une fonction de distance PNP avec hubs :

$$d_{PNP, \mathcal{A}}(\theta, \tilde{\theta}) := \frac{1}{2}(d_{PNP}(\theta, \mathcal{A}(\theta), \tilde{\theta}) + d_{PNP}(\theta, \mathcal{A}(\tilde{\theta}), \tilde{\theta})) \quad (10)$$

Cette distance est bien symétrique et nous permet de calculer l'ensemble des voisinages des θ (simplement en calculant toute les distance inter- θ) en ayant besoin de calculer uniquement les $M(h)$ pour tout $h \in \mathcal{H}$.

En choisissant $\mathcal{H}$ tel que l'approximation (7) soit valable dans les hubs mais que $|\mathcal{H}| \ll |\Theta|$ nous pouvons donc déterminer le graphe des plus proches voisins de manière efficace.

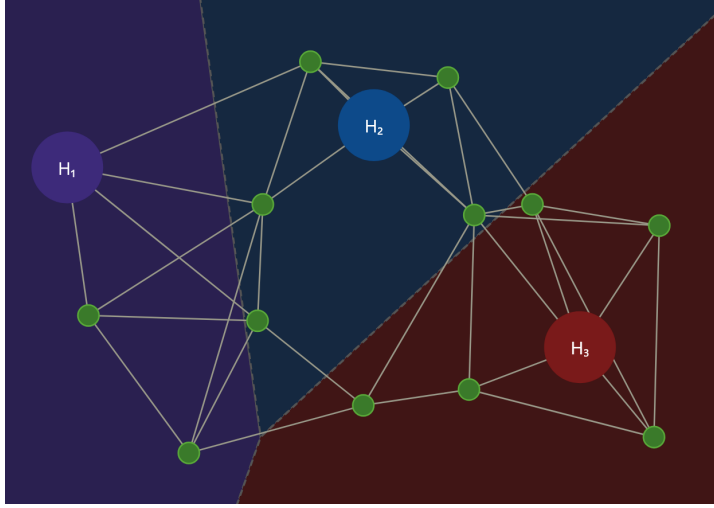


FIGURE 5 – Exemple d'allocation des hubs : on observe que la séparation de l'espace obtenue est uniquement basé sur les distances aux hubs. De plus on pourrait juger que le placement du hub 1 n'est pas le plus judicieux pour assurer la condition (7)

3.2.2 Choix des hubs

L'expression établie en (10) n'est valable que lorsque l'approximation (7) est valable dans les hubs, nous avons donc deux nouvelles problématiques :

- Comment juger de la qualité de $\mathcal{H}$?
- Comment déterminer $\mathcal{H}$?

Pour la première nous allons nous intéresser à la distribution des θ dans les hubs, en particulier les distances pour tout $h \in \mathcal{H}$ entre h et $\theta \in \mathcal{R}(h)$: si la distance moyenne est du même ordre que la déviation standard pour l'ensemble des hubs alors nous choisirons de valider ce $\mathcal{H}$.

La deuxième problématique est plus complexe, en effet nous devons déterminer $\mathcal{H}$ sans avoir recours aux $M(\theta)$ (nous voulons justement ne pas les calculer) et donc sans avoir recours à l'ensemble des distances entre les θ . De plus le fait de choisir un point dans un espace de très haute dimension (plus de 10^5) peut mener à des phénomènes contre-intuitif sur la distribution des points et la manière de se représenter et de trouver des groupes de points rapprochés.

Le choix de la distribution complète des hubs en une étape ne semblant pas possible nous nous sommes orienté vers un algorithme itératif pour lequel le problème se réduit donc à :

soit $k \in \mathbf{N}$, $\mathcal{H} \subset \Theta$ tel que $|\mathcal{H}| = k$, trouver $\tilde{h} \in \Theta \setminus \mathcal{H}$ le meilleur candidat pour être le hub $k+1$ (11)

En initialisant avec un ou deux hubs choisis aléatoirement parmi Θ .

Solution 1 : Critère de l'inégalité triangulaire

Une première solution fut de choisir comme hub le point ne respectant le moins l'inégalité triangulaire (approximation de ses distance la moins précise), on s'intéresse donc au point qui maximise le critère suivant :

$$C(\theta, \theta_1, \theta_2) = \frac{d_{PNP}(\theta_1, \mathcal{A}(\theta), \theta) + d_{PNP}(\theta, \mathcal{A}(\theta), \theta_2)}{d_{PNP}(\theta_1, \mathcal{A}(\theta_1), \theta_2) + d_{PNP}(\theta_2, \mathcal{A}(\theta_2), \theta_1)} \quad (12)$$

l'algorithme (1) fut implémenté, malheureusement la nécessité de tester toute les combinaisons de paires de hub et de candidats puis pour chacune d'entre elles de calculer les distance avec tous les points dans l'intersection des hubs (quadruple boucle for) engendre une complexité très importante

(en particulier à la suite de l'initialisation avec un nombre faible de hubs mais chacun de cardinal important).

Pour y remédier nous avons essayer de limiter le choix des combinaison à un sous ensemble de celles ci choisies aléatoirement de manière uniforme, cependant cela dégrade trop la qualité des résultats finaux (distribution des allocations très inégales). Aucun compromis ne fut satisfaisant nous avons donc décidé de ne pas garder cette méthode.

Algorithm 1 Le point choisi comme hub est celui qui maximise le critère C , nous décrivons une étape du processus itératif

Require: $ListePaires = [(h_1, h_2), \dots]$ Paires de hubs dans lesquels nous cherchons un nouveau candidat

Require: $\mathcal{H}_n$ avec au moins deux hubs

Require: $\mathcal{A}_n$

Require: $\mathcal{R}_n$

Ensure: $\mathcal{H}_{n+1}$ qui contient un hub supplémentaire

Ensure: $\mathcal{A}_{n+1}$

Ensure: $\mathcal{R}_{n+1}$

```

for  $(h, \tilde{h})$  in  $ListePaires_n$  do
  for  $\theta$  in  $\mathcal{R}_n(h) \cup \mathcal{R}_n(\tilde{h})$  do
     $R_\theta \leftarrow 0$ 
    for  $\theta_1, \theta_2$  in  $\mathcal{R}_n(h), \mathcal{R}_n(\tilde{h})$  do
       $R_\theta \leftarrow R_\theta + C(\theta, \theta_1, \theta_2)$  ▷  $C$  tel que définie en (12)
    end for
  end for
end for
 $\theta_{choisi} \leftarrow \arg \max_{\theta} R_\theta$ 
 $\mathcal{H}_{n+1} \leftarrow \mathcal{H}_n + \theta_{choisi}$ 
Compute  $\mathcal{A}_{n+1}$  and  $\mathcal{R}_{n+1}$  from  $\mathcal{H}_{n+1}$  ▷ Selon les définitions explicités en (3.2.1)
 $ListePaires_{n+1} \leftarrow \emptyset$ 
for  $\theta$  in  $\Theta$  do
  if  $\mathcal{A}_n(\theta) \neq \mathcal{A}_{n+1}(\theta)$  then
     $ListePaires_{n+1} \leftarrow ListePaires_{n+1} + (\mathcal{A}_n(\theta), \mathcal{A}_{n+1}(\theta))$ 
  end if
end for
end for
```

Solution 2 : Tirage dans le hub le plus peuplé

Une seconde solution est basé sur l'intuition qu'une bonne allocation de hub va répartir les points. On cherche donc à chaque itération à séparer le hub contenant le plus de points en deux hubs plus petits. Pour ce faire on choisi de manière uniforme un point dans la région de ce hub, en excluant les points les plus proches et les plus lointains du centre. On initialise donc avec un hub choisi aléatoirement parmi Θ .

Cette méthode fut implémenté dans l'algorithme (2) et les hubs résultants ont présentés plusieurs caractéristiques recherchées : moyennes et écarts type des distances dans les hubs sont de même ordres de grandeurs, absence d'un hub contenant la majorité des points comme précédemment.

Cependant la part d'aléatoire de l'algorithme lui fait choisir régulièrement un mauvais candidat qui va être l'unique membre de son hubs. Auquel cas le calcul de $M(h)$ ne peut pas être utilisé pour calculer plusieurs distances, il n'a donc aucun intérêt dans notre cas.

Pour empêcher cela nous avons choisi d'employer une méthode naïve : lors des itérations nous supprimons régulièrement les hubs isolés ($|\mathcal{R}(h)| = 1$) avant de les réallouer au hubs restants.

Nous obtenons ainsi une distribution de hubs ne contenant que des hubs significatifs ($|\mathcal{R}(h)| > 1$), mais pour atteindre le même nombre de hubs il faut compter un temps de calcul 3 à 4 fois plus long (heuristique pour 500 hubs)

Algorithm 2 Le point choisi comme hub est tiré de manière uniforme un point dans la région du hub le plus peuplé, nous décrivons une étape du processus itératif

Require: $\mathcal{H}_n$ avec au moins deux hubs

Require: $\mathcal{A}_n$

Require: $\mathcal{R}_n$

Ensure: $\mathcal{H}_{n+1}$ qui contient un hub supplémentaire

Ensure: $\mathcal{A}_{n+1}$

Ensure: $\mathcal{R}_{n+1}$

$h_{dominant} \leftarrow \arg \max_{h \in \mathcal{H}_n} |\mathcal{H}_n(h)|$

$L_{distances} \leftarrow \emptyset$

for $\theta \in \mathcal{R}(h_{dominant})$ **do**

 Add $d_{PNP, \mathcal{A}_n}(\theta, h_{dominant})$ to $L_{distances}$

end for

Sort $L_{distances}$ in descending order

$i_{10\%} \leftarrow \lfloor 0.1 |L_{distances}| \rfloor$

$i_{70\%} \leftarrow \lfloor 0.7 |L_{distances}| \rfloor$

Randomly pick θ_{choisi} in $L_{distances}[i_{10\%} : i_{70\%}]$

$\mathcal{H}_{n+1} \leftarrow \mathcal{H}_n + \theta_{choisi}$

Compute $\mathcal{A}_{n+1}$ and $\mathcal{R}_{n+1}$ from $\mathcal{H}_{n+1}$

▷ Selon les définitions explicités en (3.2.1)

3.3 Création du graphe

Une fois la fonction de distance définie pour tous les θ (10) et la répartition des hubs qui lui est associée) nous pouvons construire le graphe des plus proches voisins de manière classique.

Celui-ci a donc en plus du paramètre K du nombre de voisins, un paramètre de contrôle du nombre de hubs N_{hubs} qui va permettre de choisir le compromis entre temps de calcul et qualité de l'approximation des mesures de distances.

Dans la suite nous avons choisi par exemple $N_{hubs} = 500$.

Le graphe de la méthode Bruteforce (graphe de référence) et celui de la méthode P-loss peuvent aussi être créés avec les distances entre les $S(\theta)$ (déjà calculés pour la première partie) ou respectivement les paramètres. Ces graphes permettent de tester la qualité de l'approximation du graphe PNP dans notre base de données exemple (à K constant pour les trois graphes).

3.4 Caractéristiques du graphe

Suite à la création de ces graphes nous nous sommes intéressés à leurs différentes caractéristiques telles que la répartition des degrés, le nombre de composantes connexes et l'influence du paramètre K sur ces caractéristiques. On note notamment à l'aide la Figure 6 que l'augmentation du nombre de voisins pris en compte pour la construction du graphe (K) permet de réduire le nombre de composantes connexes. Lorsque $k \geq 23$ le graphe réalisé à l'aide de la méthode PNP présente une unique composante connexe, il sera donc possible de l'utiliser pour réaliser n'importe quel parcours d'un son à un autre. Pour des valeurs inférieures à 23 le graphe restera utilisable pour créer des chemins entres deux sons présents dans le même composante connexe ou créer des marches dans différentes composantes.

La figure 7 présente la répartition des degrés pour ces différents graphes que nous avons réalisés. Le graphe de référence présente un nombre important de nœuds isolés. Cette caractéristique provient d'erreurs numériques lors du calcul de JTFS que nous n'avons malheureusement pas réussi à contourner. Par ailleurs on remarque que l'impact des approximations est très important et risque donc d'avoir des répercussions non négligeables sur les résultats finaux. On note notamment pour le graphe PNP que plus le degré est faible (se rapprochant de K) plus le nombre de nœuds ayant ce degré est élevé. Cette observation reste valable avec différentes valeurs prises pour K (Figure 7). Ceci est probablement du à la méthode de construction des Hubs qui ne parvient pas à donner une approximation satisfaisante pour tout l'espace. Il est en revanche intéressant de noter que le nombre de nœuds diminue lorsque le degré augmente, comme pour le graphe de référence.

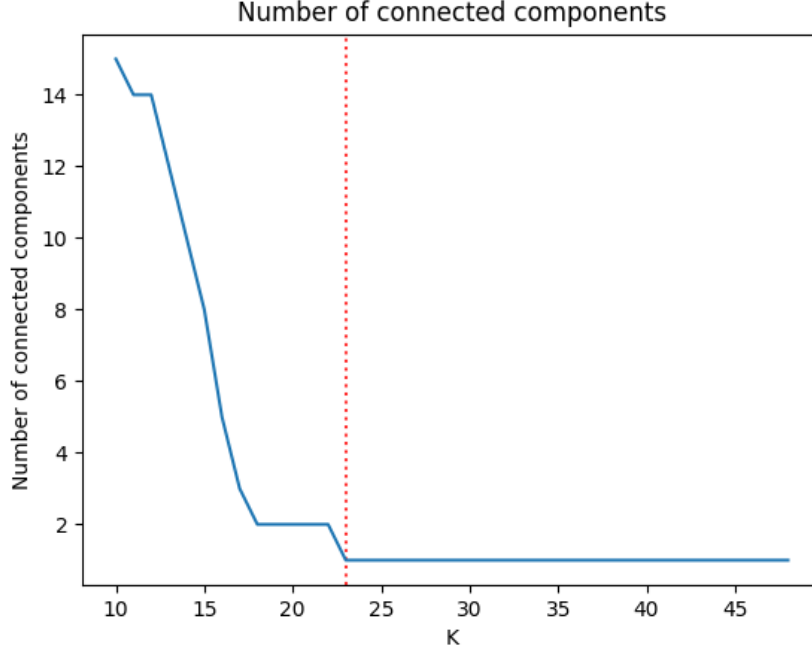


FIGURE 6 – Nombre de composantes connexes présentes dans le graphe réalisé à l’aide de la méthode PNP pour différentes valeurs de K . Le nombre de composantes connexes diminue lorsque K augmente. On observe une unique composante lorsque $K \geq 23$.

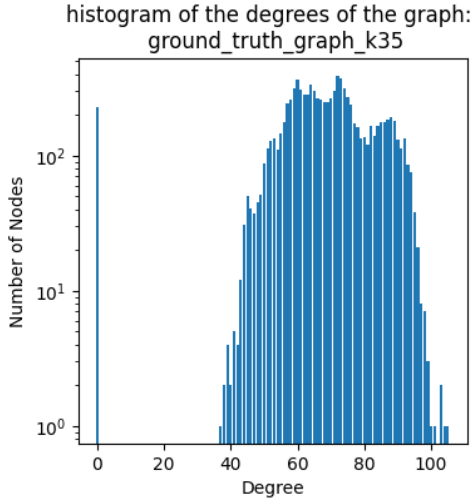
3.5 Clustering

Pour nous permettre de déterminer une valeur optimale du paramètre K nous avons envisagé de réaliser un clustering sur le graphe et d’évaluer la pertinence de ce clustering à l’aide de l’indice de modularité[6]. Nous espérons à l’aide de cette méthode, réussir à segmenter le graphe en régions perceptuelles (instrument à bois, métal...) ce qui confirmerait la bonne représentation de l’espace sonore par la construction de ce graphe avec une certaine valeur du paramètre K .

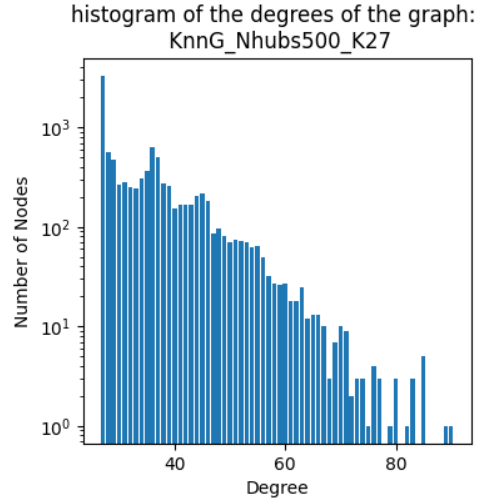
Nous avons donc testé trois méthodes de clustering pour des valeurs de k comprises entre 10 et 48. La première méthode envisagée est la méthode de clustering de Louvain [7]. Cette méthode itérative maximise l’indice de modularité en modifiant les noeuds compris dans les clusters créés. La deuxième méthode est la méthode de Leiden qui est une amélioration du clustering de Louvain [8] permettant de garantir que tous les clusters soient connexes et de conserver les clusters de petites tailles que l’algorithme de Louvain a tendance à faire disparaître au cours des itérations. Finalement nous avons aussi envisagé la méthode de walktrap [9] proposant une approche par marches aléatoires. Le principe est de réaliser de courtes marches aléatoires en supposant que ces marches ont de fortes chances de rester dans un même cluster. Les résultats de ces méthodes sont décrits en Figure 8.

Il est intéressant de noter que la méthode de walktrap semble avoir un score très différent des autres tout en ayant un nombre de clusters du même ordre de grandeur. Ceci semble provenir du fait que cette méthode ne prend pas en compte les distances calculées alors que ces dernières ont une influence sur le calcul d’indice de modularité. Et que les algorithmes de Louvain et Leiden sont très proches.

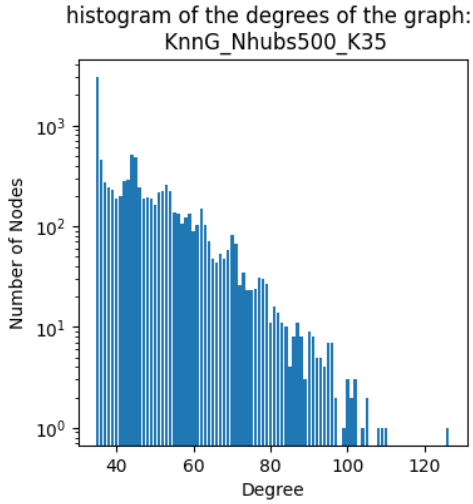
Après avoir réalisé le clustering avec ces différentes méthodes, nous ne sommes malheureusement pas parvenu à segmenter le graphe en clusters perceptuels. En effet la seule donnée de la distance entre chaque nœud ne semble pas suffisante étant donnée la complexité de cette problématique.



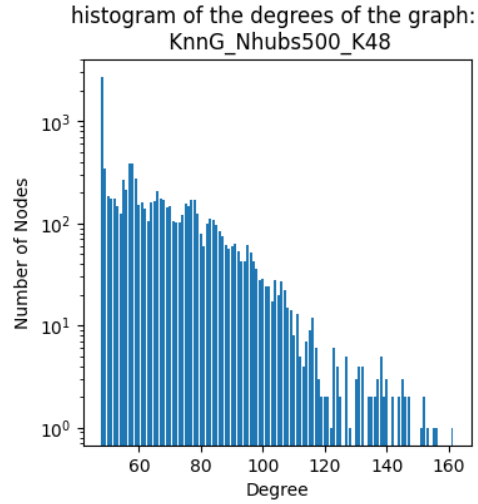
(a) Graphe de référence, $K = 35$



(b) Graphe PNP, $K = 27$



(c) Graphe PNP, $K = 35$



(d) Graphe PNP, $K = 48$

FIGURE 7 – Histogramme des degrés pour différents graphes. En haut à gauche le graphe de référence présente un nombre important de nœuds isolés dû à des erreurs numériques lors du calcul de la JTFS. On observe une différence notable entre la répartition des degrés entre le graphe de référence (en haut à gauche) et le graphe PNP construit avec différentes valeurs de K .

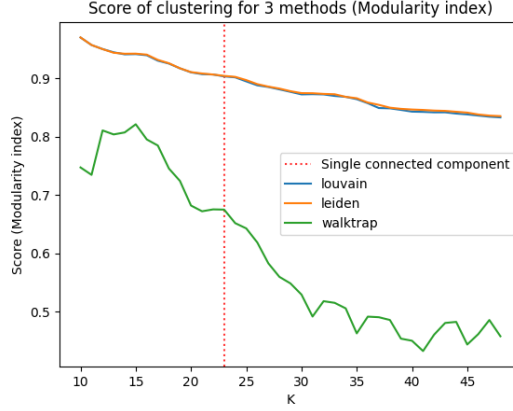


FIGURE 8 – Scores pour différentes méthodes de clustering. Les méthodes de Louvain et de Leiden donnent des résultats similaires alors que la méthode de walktrap possède un score bien inférieur.

4 Applications

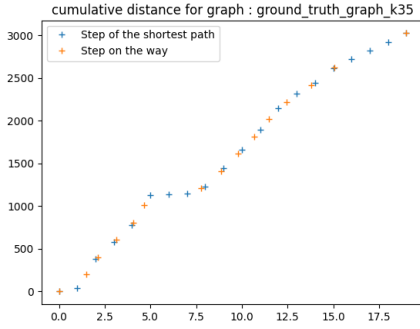
La construction de ce graphe ouvre maintenant des possibilités de création et d'application.

4.1 Chemin entre deux sons de référence

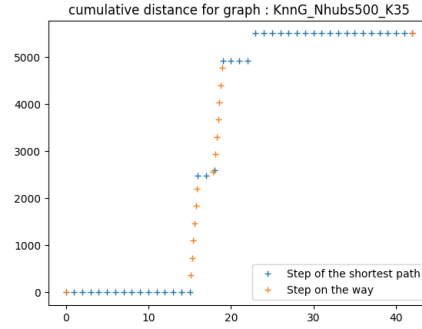
Le premier cas d'application est de partir d'un premier son de référence pour arriver à un autre et construire un chemin plus ou moins continu perceptuellement entre les deux. Pour réaliser ce chemin on calcule tout d'abord le plus court chemin à l'aide de l'algorithme de Dijkstra [10]. Ce parcours contient des distances irrégulières entre les points. Pour obtenir un chemin perceptuellement satisfaisant il faut donc trouver un moyen de régulariser ces distances. Pour ce faire nous considérons la distance totale entre le point initial et le point final donné par le parcours. Nous divisons cette quantité par le nombre d'étapes que nous souhaitons pour le chemin, puis nous pouvons déterminer pour chaque étape les deux points entre lesquels point de passage doit se trouver. Pour finalement calculer les paramètres associés au son de cette étape nous considérons que l'algorithme JTFS est localement linéaire, ce qui permet de trouver les paramètres de l'étape recherchée avec une combinaison linéaire entre les paramètres des deux points. Le résultat de cette méthode est montrée en Figure 9.

4.2 Marche aléatoire

Une application intéressante est aussi de réaliser une marche aléatoire sur le graphe suivant les distances perceptuelles entre les sons voisins dans le graphe. Pour cela on utilise la méthode de marche aléatoire de networkx [11] (*generate_random_path*). Notre méthode est particulièrement adaptée à cette application car l'approximation utilisée est fiable localement et surtout conserve en grande partie l'ordre des plus proches voisins. Pour essayer de visualiser le parcours réalisé nous avons construit un graphe qui représente ce parcours. Pour cela nous avons extrait les nœuds et utilisés pour le parcours et toutes les arêtes reliant ces nœuds. Cela permet de voir si le parcours contient différents clusters, ce qui pourrait s'apparenter à différentes familles de sons. On remarque notamment par la Figure 10s que le parcours réalisé dans le graphe de référence présente deux familles de sons alors que le parcours dans le graphe construit à partir du parcours réalisé dans le graphe construit à l'aide de notre approximation semble très régulier. On remarque de plus que ce graphe contient un nombre de nœuds largement inférieur aux 30 nœuds attendus pour le parcours. Ces nœuds sont donc présents un grand nombre de fois et le parcours ne parvient donc pas à sortir de cette liste de nœuds et gravite autour d'elle.

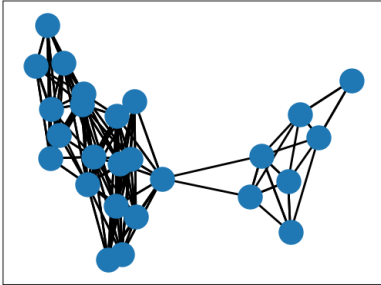


(a) Pour un parcours au sein du graphe de référence

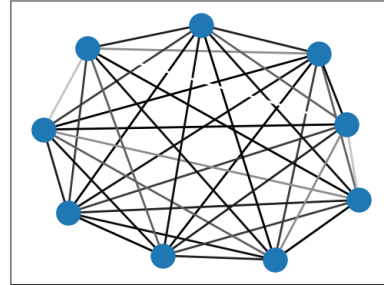


(b) Pour un parcours au sein du graphe PNP

FIGURE 9 – Distance cumulée pour un chemin entre deux sons réalisé sur le graphe de référence (à gauche) et le graphe PNP (à droite). Les points oranges représentent le chemin final, les points bleus représentent les points du chemin le plus court. Le chemin construit par les points orange possède des distances égales entre chaque point pour créer le chemin le plus linéaire au sens de la perception humaine.



(a) Graphe issu d'une la marche aléatoire de 30 pas dans le graphe de référence



(b) Graphe issu d'une la marche aléatoire de 30 pas dans le graphe PNP

FIGURE 10 – Graphes issus de marches aléatoires de 30 pas dans le graphe de référence (à gauche) et dans le graphe PNP (à droite) avec $K = 35$. La marche aléatoire dans le graphe PNP passe par un nombre restreint de nœuds fortement interconnectés.

5 Conclusion sur les résultats finaux

Notre projet a tout d'abord révélé les performances de l'approximation de la distance riemannienne pour la détermination des plus proches voisins. Cette méthode permet une grande accélération de l'algorithme tout en conservant des résultats satisfaisants pour un nombre de voisins inférieurs à 70. De plus nous avons montré qu'il est possible de construire un graphe des plus proches voisins à partir des distances calculées avec une méthode par placement de Hubs permettant de ne calculer qu'un certain nombre de matrices M . Nous avons aussi révélé les possibilités créatives de l'utilisation de la métrique JTFS en déterminant plusieurs suggestions à partir d'un son de référence qui peut donc compléter la réponse unique donné par une solution basée sur le machine learning par exemple. Par ailleurs, les applications présentées dans ce projet fournissent des résultats perceptuellement intéressants pour la création musicale lorsqu'elles sont réalisées à partir du graphe de référence. Cela permet donc de mettre en lumière la robustesse de JTFS. Il est cependant important de noter que l'approximation de la métrique Riemannienne est performante pour déterminer les k plus proches voisins d'un son uniquement pour une valeur de k très faible ce qui pose un problème fondamental pour la création du graphe des plus proches voisins nécessitant une valeur de K supérieure à 24. De plus la méthode de construction du graphe semble perfectible pour obtenir des caractéristiques plus proches du graphe de référence.

Références

- [1] Han HAN, Vincent LOSTANLEN et Mathieu LAGRANGE. *Perceptual-Neural-Physical Sound Matching*. 2023. arXiv : 2301.02886 [cs.SD]. URL : <https://arxiv.org/abs/2301.02886>.
- [2] Han HAN et Vincent LOSTANLEN. *Perceptual Neural Physical Sound Matching*. https://github.com/lylyhan/perceptual_neural_physical. 2023.
- [3] Han HAN, Vincent LOSTANLEN et Mathieu LAGRANGE. *Apprentissage de variété riemannienne pour l'analyse-synthèse de signaux non stationnaires*. Grenoble, France, août 2023. URL : <https://hal.science/hal-04145714>.
- [4] John MURADELI et al. *Differentiable Time-Frequency Scattering on GPU*. 2022. arXiv : 2204.08269 [cs.SD]. URL : <https://arxiv.org/abs/2204.08269>.
- [5] Mathieu ANDREUX et al. *Kymatio : Scattering Transforms in Python*. 2022. arXiv : 1812.11214 [cs.LG]. URL : <https://arxiv.org/abs/1812.11214>.
- [6] Song YANG. *Networks : An Introduction by MEJ Newman : Oxford, UK : Oxford University Press. 720 pp., \$85.00*. 2013.
- [7] Vincent D BLONDEL et al. “Fast unfolding of communities in large networks”. In : *Journal of statistical mechanics : theory and experiment* 2008.10 (2008), P10008.
- [8] Vincent A TRAAG, Ludo WALTMAN et Nees Jan VAN ECK. “From Louvain to Leiden : guaranteeing well-connected communities”. In : *Scientific reports* 9.1 (2019), p. 5233.
- [9] Pascal PONS et Matthieu LATAPY. “Computing Communities in Large Networks Using Random Walks”. In : *Computer and Information Sciences - ISCIS 2005*. Sous la dir. de pInar YOLUM et al. Berlin, Heidelberg : Springer Berlin Heidelberg, 2005, p. 284-293.
- [10] Edsger W DIJKSTRA et al. *A discipline of programming*. T. 613924118. prentice-hall Englewood Cliffs, 1976.
- [11] Jing ZHANG et al. “Panther : Fast top-k similarity search on large networks”. In : *Proceedings of the 21th ACM SIGKDD international conference on knowledge discovery and data mining*. 2015, p. 1445-1454.